



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения

**ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5
по дисциплине
«Технологии виртуализации клиент-серверных приложений»**

Выполнил студент группы ИКБО-16-22

Грибков А.С.

Принял преподаватель кафедры ИиППО

Волков М.Ю.

Практические работы выполнены

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

Москва
2025

СОДЕРЖАНИЕ

1 Цель работы	3
2 Краткая теоретическая часть	4
2.1 Kubernetes	4
2.2 Minikube.....	4
2.3 kubectl	5
2.4 Pod	5
2.5 Deployment	6
2.6 Service	6
3 Ход работы	8
3.1 Установка и запуск Minikube	8
3.2 Подготовка Docker-образа	9
3.3 Создание Deployment	11
3.4 Просмотр информации о кластере	12
3.5 Создание Service для доступа к приложению	14
3.6 Запуск и тестирование сервиса	15
3.7 Работа с дополнениями Minikube	16
3.8 Работа с Kubernetes Dashboard	17
3.9 Удаление ресурсов и остановка кластера.....	19
4 Вывод.....	21
5 Ответы на вопросы к практической работе	22
5.1 Назовите виды контроллеров в Kubernetes	22
5.2 Как называется командная строка в Kubernetes?.....	23
5.3 Что такое под?.....	23
5.4 Назовите 2 типа ресурсов, из которых состоит кластер Kubernetes	24
5.5 Чем Kubernetes отличается от Docker Swarm?.....	25
6 Список использованной литературы	28

1 ЦЕЛЬ РАБОТЫ

Целью данной практической работы является изучение системы оркестрации контейнеров Kubernetes и приобретение практических навыков работы с локальной средой разработки Minikube.

Основные задачи работы:

- Установить и настроить Minikube для локального развертывания Kubernetes кластера
- Создать простое веб-приложение на Node.js и упаковать его в Docker-образ
- Развернуть приложение в Kubernetes кластере с использованием Deployment
- Изучить основные команды kubectl для управления кластером и просмотра его состояния
- Настроить доступ к приложению через Service типа NodePort
- Освоить работу с дополнениями Minikube (addons)
- Познакомиться с веб-интерфейсом Kubernetes Dashboard

2 КРАТКАЯ ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

2.1 Kubernetes

Kubernetes — это открытая платформа для автоматизации развертывания, масштабирования и управления контейнеризированными приложениями. Kubernetes группирует контейнеры, составляющие приложение, в логические единицы для упрощения управления и обнаружения.

Основные компоненты архитектуры Kubernetes:

- **Control Plane** — управляющий слой кластера, отвечающий за принятие глобальных решений о кластере и обнаружение событий кластера.
- **Nodes** — рабочие узлы кластера, на которых выполняются контейнеризированные приложения.
- **Pods** — минимальная развертываемая единица в Kubernetes, представляющая один или несколько контейнеров.
- **Deployments** — описывают желаемое состояние приложения и управляют созданием и обновлением Pods.
- **Services** — абстракция, определяющая логический набор Pods и политику доступа к ним.

2.2 Minikube

Minikube — это инструмент для локального запуска Kubernetes. Minikube запускает одноузловой кластер Kubernetes на локальном компьютере для пользователей, желающих изучить Kubernetes или разрабатывать приложения локально.

Основные возможности Minikube:

- Поддержка нескольких драйверов виртуализации (Docker, VirtualBox, KVM и др.)
- Поддержка Kubernetes dashboard
- Работа с различными версиями Kubernetes
- Поддержка дополнений (addons) для расширения функциональности
- Локальный реестр Docker для использования собственных образов

2.3 kubectl

kubectl — это инструмент командной строки для взаимодействия с кластером Kubernetes. Он позволяет запускать команды для управления кластером, развертывания приложений, проверки и управления ресурсами кластера, а также просмотра логов.

Основные группы команд kubectl:

- Управление ресурсами (create, apply, delete)
- Просмотр информации о ресурсах (get, describe)
- Работа с приложениями (scale, expose, rollout)
- Отладка и диагностика (logs, exec, port-forward)

2.4 Pod

Pod — это наименьшая разворачиваемая единица в Kubernetes, которая может быть создана и управляема. Pod представляет собой группу из одного или нескольких контейнеров со shared storage и network ресурсами, а также спецификацией по запуску контейнеров.

Характеристики Pod:

- Все контейнеры в Pod разделяют IP-адрес и порты

- Контейнеры в Pod могут обмениваться данными через localhost
- Pod эфемерны — они могут быть созданы и уничтожены по мере необходимости
- Обычно Pod управляется контроллерами высокого уровня (например, Deployment)

2.5 Deployment

Deployment — это объект Kubernetes, который описывает желаемое состояние приложения. Deployment управляет созданием и обновлением экземпляров приложения (Pods) и обеспечивает декларативное обновление приложений.

Основные функции Deployment:

- Автоматическое создание и масштабирование Pods
- Откат к предыдущим версиям приложения
- Rolling updates — обновление приложений без простоя
- Самовосстановление — автоматический перезапуск неисправных Pods

2.6 Service

Service — это абстракция, которая определяет логический набор Pods и политику доступа к ним. Service обеспечивает стабильную сетевую точку входа для доступа к динамическому набору Pods.

Типы Service:

- **ClusterIP** — предоставляет доступ к Service только внутри кластера
- **NodePort** — открывает Service на указанном порту каждого узла кластера
- **LoadBalancer** — создает внешний балансировщик нагрузки

- **ExternalName** — отображает Service на DNS-имя

3 ХОД РАБОТЫ

3.1 Установка и запуск Minikube

Первым шагом необходимо установить Minikube и запустить локальный Kubernetes кластер. После установки выполняется команда для запуска кластера:

```
alexgribkov@i1113971732 report % minikube start
🐳 minikube v1.37.0 on Darwin 15.6.1 (arm64)
🔧 Using the docker driver based on existing profile
👍 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.48 ...
🏃 Updating the running docker "minikube" container ...
🐳 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔍 Verifying Kubernetes components...
  ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
  ▪ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.0
  ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
  ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
💡 Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

🌟 Enabled addons: storage-provisioner, metrics-server, default-storageclass, dashboard
🏃 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Рисунок 3.1 — Запуск Minikube кластера

После успешного запуска кластера проверяется его статус:

```
alexgribkov@i1113971732 report % minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Рисунок 3.2 — Проверка статуса Minikube

3.2 Подготовка Docker-образа

Для развертывания в Kubernetes необходимо создать простое веб-приложение и упаковать его в Docker-образ. Приложение представляет собой простой HTTP-сервер на Node.js.

Листинг 3.1 — Код файла server.js

```
var http = require('http');
var handleRequest = function(request, response) {
  console.log('Получени запрос на URL:' + request.url);
  response.writeHead(200); response.end('Hello World!'); }; var www
= http.createServer(handleRequest); www.listen(8080);
```

Для упаковки приложения создается Dockerfile:

Листинг 3.2 — Содержимое Dockerfile

```
FROM node:6.14.2 EXPOSE 8080 COPY server.js . CMD ["node",
"server.js"]
```

Для использования локального Docker-образа в Minikube необходимо настроить окружение Docker:

```
alexgribkov@i113971732 report % eval $(minikube docker-env)
alexgribkov@i113971732 report % echo "Docker environment configured for Minikube"
Docker environment configured for Minikube
```

Рисунок 3.3 — Настройка Docker окружения для Minikube

Выполняется сборка Docker-образа с необходимым именем:

```
alexgribkov@i113971732 practical-work-5 % docker build -t gribkov-ikbo-16-22-obraz .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
           Install the buildx component to build images with BuildKit:
           https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  770.6kB
Step 1/4 : FROM node:6.14.2
---> ed66c0b1c9c8
Step 2/4 : EXPOSE 8080
---> Using cache
---> 6469c04178c8
Step 3/4 : COPY server.js .
---> Using cache
---> f668440050f9
Step 4/4 : CMD ["node", "server.js"]
---> Using cache
---> 02ce1c860307
Successfully built 02ce1c860307
Successfully tagged gribkov-ikbo-16-22-obraz:latest
```

Рисунок 3.4 — Сборка Docker-образа

Проверяется наличие созданного образа:

```
alexgribkov@i113971732 practical-work-5 % docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED
gribkov/journal-server	v1	6882db6f9fdd	About an hour ago
gribkov/static-files	v1	f1764856e06b	About an hour ago
gribkov-ikbo-16-22-obraz	latest	02ce1c860307	3 hours ago
registry.k8s.io/ingress-nginx/controller	<none>	5ed383cb88c3	2 months ago
registry.k8s.io/ingress-nginx/kube-webhook-certgen	<none>	c67c707f59d8	2 months ago
registry.k8s.io/kube-apiserver	v1.34.0	d291939e9940	2 months ago
registry.k8s.io/kube-controller-manager	v1.34.0	996be7e86d9b	2 months ago
registry.k8s.io/kube-scheduler	v1.34.0	a25f5ef9c34c	2 months ago
registry.k8s.io/kube-proxy	v1.34.0	6fc32d66c141	2 months ago
registry.k8s.io/etcd	3.6.4-0	a1894772a478	3 months ago
registry.k8s.io/metrics-server/metrics-server	<none>	bc6c1e09a843	4 months ago
registry.k8s.io/pause	3.10.1	d7b100cd9a77	5 months ago
registry.k8s.io/coredns/coredns	v1.12.1	138784d87c9c	7 months ago
redis	5-alpine	8ab49366732f	3 years ago
kubernetesui/dashboard	<none>	20b332c9a70d	3 years ago
kubernetesui/metrics-scraper	<none>	a422e0e98235	3 years ago
gcr.io/k8s-minikube/storage-provisioner	v5	ba04bb24b957	4 years ago
node	6.14.2	ed66c0b1c9c8	7 years ago

Рисунок 3.5 — Проверка созданного Docker-образа

3.3 Создание Deployment

Для развертывания приложения в Kubernetes создается манифест Deployment, который описывает желаемое состояние приложения:

Листинг 3.3 — Содержимое файла deployment.yaml

```
apiVersion: apps/v1 kind: Deployment metadata: name: gribkov-ikbo-16-22 spec: replicas: 1 selector: matchLabels: app: gribkov-ikbo-
```

```
16-22 template: metadata: labels: app: gribkov-ikbo-16-22 spec:
containers: - name: gribkov-ikbo-16-22-container image: gribkov-
ikbo-16-22-obraz imagePullPolicy: Never ports: - containerPort:
8080
```

Применяется созданный манифест для создания Deployment в кластере:

```
alexgribkov@i113971732 practical-work-5 % kubectl apply -f deployment.yaml
deployment.apps/gribkov-ikbo-16-22 created
```

Рисунок 3.6 — Применение манифеста Deployment

3.4 Просмотр информации о кластере

После создания Deployment необходимо проверить состояние развернутых ресурсов. Просматривается список Deployments:

```
alexgribkov@i113971732 practical-work-5 % kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
fileserver          2/2     2             2           24m
frontend            2/2     2             2           3s
gribkov-ikbo-16-22  1/1     1             1           17s
```

Рисунок 3.7 — Список Deployments в кластере

Просматривается список Pods:

```
alexgribkov@i113971732 practical-work-5 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
fileserver-75576c7b6c-cbztd         1/1     Running   1 (6m34s ago)  25m
fileserver-75576c7b6c-tm5bz         1/1     Running   1 (6m34s ago)  25m
frontend-667d95dc97-bsjbs           1/1     Running   0           22s
frontend-667d95dc97-c9csr           1/1     Running   0           22s
frontend-6d7786bc84-75kv8           1/1     Terminating 0           3m8s
frontend-6d7786bc84-xfhf2           1/1     Terminating 0           3m7s
gribkov-ikbo-16-22-776f997d54-b52rv 1/1     Running   0           36s
redis-0                             1/1     Running   1 (6m24s ago)  25m
redis-1                             1/1     Running   1 (6m24s ago)  25m
redis-2                             1/1     Running   1 (6m24s ago)  25m
```

Рисунок 3.8 — Список Pods в кластере

Просматриваются события кластера для отслеживания процесса развертывания:

```

58m      Normal    Scheduled                pod/redis-2
      Successfully assigned default/redis-2 to minikube
58m      Normal    Pulled                  pod/redis-2
      Container image "redis:5-alpine" already present on machine
58m      Normal    Created                 pod/redis-2
      Created container: redis
58m      Normal    Started                 pod/redis-2
      Started container redis
49m      Normal    Killing                 pod/redis-2
      Stopping container redis
25m      Warning   FailedScheduling        pod/redis-2
      0/1 nodes are available: pod has unbound immediate PersistentVolumeClaims.
not found
25m      Warning   FailedScheduling        pod/redis-2
      0/1 nodes are available: pod has unbound immediate PersistentVolumeClaims.
not found
25m      Normal    Scheduled                pod/redis-2
      Successfully assigned default/redis-2 to minikube
6m27s    Normal    Pulled                  pod/redis-2
      Container image "redis:5-alpine" already present on machine
6m26s    Normal    Created                 pod/redis-2
      Created container: redis
6m24s    Normal    Started                 pod/redis-2
      Started container redis
6m36s    Normal    SandboxChanged          pod/redis-2
      Pod sandbox changed, it will be killed and re-created.
67m      Normal    SuccessfulCreate         statefulset/redis
      create Claim data-redis-0 Pod redis-0 in StatefulSet redis success
67m      Normal    SuccessfulCreate         statefulset/redis
      create Pod redis-0 in StatefulSet redis successful
59m      Normal    SuccessfulCreate         statefulset/redis
      create Claim data-redis-1 Pod redis-1 in StatefulSet redis success
59m      Normal    SuccessfulCreate         statefulset/redis
      create Pod redis-1 in StatefulSet redis successful
58m      Normal    SuccessfulCreate         statefulset/redis
      create Claim data-redis-2 Pod redis-2 in StatefulSet redis success
58m      Normal    SuccessfulCreate         statefulset/redis
      create Pod redis-2 in StatefulSet redis successful
25m      Normal    SuccessfulCreate         statefulset/redis
      create Claim data-redis-0 Pod redis-0 in StatefulSet redis success
25m      Normal    SuccessfulCreate         statefulset/redis
      create Pod redis-0 in StatefulSet redis successful
25m      Normal    SuccessfulCreate         statefulset/redis
      create Claim data-redis-1 Pod redis-1 in StatefulSet redis success
25m      Normal    SuccessfulCreate         statefulset/redis
      create Pod redis-1 in StatefulSet redis successful
25m      Normal    SuccessfulCreate         statefulset/redis
      create Claim data-redis-2 Pod redis-2 in StatefulSet redis success
25m      Normal    SuccessfulCreate         statefulset/redis
      create Pod redis-2 in StatefulSet redis successful

```

Рисунок 3.9 — События кластера Kubernetes

Просматривается конфигурация kubectl:

```
alexgribkov@i113971732 practical-work-5 % kubectl config view
apiVersion: v1
clusters:
- cluster:
  certificate-authority: /Users/alexgribkov/.minikube/ca.crt
  extensions:
  - extension:
    last-update: Sun, 16 Nov 2025 00:15:00 MSK
    provider: minikube.sigs.k8s.io
    version: v1.37.0
    name: cluster_info
  server: https://127.0.0.1:32781
  name: minikube
contexts:
- context:
  cluster: minikube
  extensions:
  - extension:
    last-update: Sun, 16 Nov 2025 00:15:00 MSK
    provider: minikube.sigs.k8s.io
    version: v1.37.0
    name: context_info
  namespace: default
  user: minikube
  name: minikube
current-context: minikube
kind: Config
users:
- name: minikube
  user:
    client-certificate: /Users/alexgribkov/.minikube/profiles/minikube/client.crt
    client-key: /Users/alexgribkov/.minikube/profiles/minikube/client.key
```

Рисунок 3.10 — Конфигурация kubectl

3.5 Создание Service для доступа к приложению

Для обеспечения доступа к приложению создается Service типа NodePort:

```
alexgribkov@i113971732 practical-work-5 % kubectl expose deployment gribkov-ikbo-16
-22 --type=NodePort --port=8080

service/gribkov-ikbo-16-22 exposed
```

Рисунок 3.11 — Создание Service типа NodePort

Просматривается список Services в кластере:

```
alexgribkov@i113971732 practical-work-5 % kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
fileserver	ClusterIP	10.96.3.237	<none>	80/TCP	26m
frontend	ClusterIP	10.110.36.158	<none>	8080/TCP	26m
gribkov-ikbo-16-22	NodePort	10.104.253.55	<none>	8080:32389/TCP	23s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	3h18m
redis	ClusterIP	10.109.90.86	<none>	6379/TCP	26m
redis-write	ClusterIP	None	<none>	6379/TCP	26m

Рисунок 3.12 — Список Services в кластере

3.6 Запуск и тестирование сервиса

Выполняется команда для открытия доступа к сервису через браузер:

```
alexgribkov@i113971732 practical-work-5 % minikube service gribkov-ikbo-16-22
```

NAMESPACE	NAME	TARGET PORT	URL
default	gribkov-ikbo-16-22	8080	http://192.168.49.2:32389

🏃 Starting tunnel for service gribkov-ikbo-16-22.

NAMESPACE	NAME	TARGET PORT	URL
default	gribkov-ikbo-16-22		http://127.0.0.1:63205

🏃 Starting tunnel for service gribkov-ikbo-16-22.
🏃 Opening service default/gribkov-ikbo-16-22 in default browser...
! Because you are using a Docker driver on darwin, the terminal needs to be open to run it.

Рисунок 3.13 — Открытие сервиса через Minikube

Проверяется работоспособность приложения путем отправки HTTP-запроса:

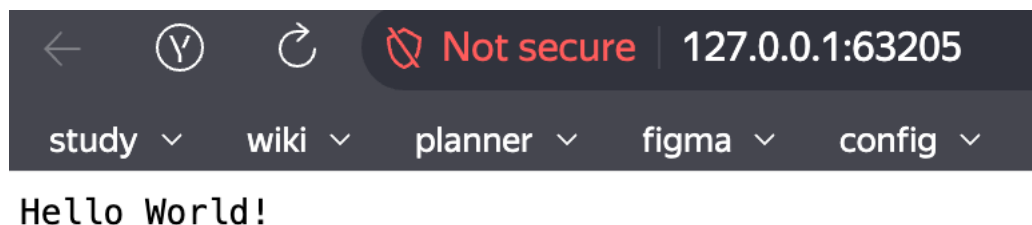


Рисунок 3.14 — Тестирование работы веб-приложения

3.7 Работа с дополнениями Minikube

Minikube предоставляет набор дополнений (addons) для расширения функциональности кластера. Просматривается список доступных дополнений:

```
alexgribkov@i113971732 practical-work-5 % minikube addons list
```

ADDON NAME	PROFILE	STATUS	MAINTAINER
ambassador	minikube	disabled	3rd party (Ambassador)
amd-gpu-device-plugin	minikube	disabled	3rd party (AMD)
auto-pause	minikube	disabled	minikube
cloud-spanner	minikube	disabled	Google
csi-hostpath-driver	minikube	disabled	Kubernetes
dashboard	minikube	enabled ✓	Kubernetes
default-storageclass	minikube	enabled ✓	Kubernetes
efk	minikube	disabled	3rd party (Elastic)
freshpod	minikube	disabled	Google
gcp-auth	minikube	disabled	Google
gvisor	minikube	disabled	minikube
headlamp	minikube	disabled	3rd party (kinvolk.io)
inaccel accel.com]]	minikube	disabled	3rd party (InAcce1 [info@in
ingress	minikube	disabled	Kubernetes
ingress-dns	minikube	disabled	minikube
inspektor-gadget	minikube	disabled	3rd party (inspektor-gadget

Рисунок 3.15 — Список дополнений Minikube

Включается дополнение ingress, которое предоставляет возможность настройки входящего трафика:


```
alexgribkov@i113971732 practical-work-5 % minikube addons enable ingress
💡 ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
💡 After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
▪ Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
🔍 Verifying ingress addon...
🌟 The 'ingress' addon is enabled
```

Рисунок 3.16 — Включение дополнения ingress

Проверяется состояние Pods и Services в системном namespace:

```
alexgribkov@i113971732 practical-work-5 % kubectl get pod,svc -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
pod/coredns-66bc5c9577-s95tk	1/1	Running	4 (10m ago)	3h21m
pod/etcd-minikube	1/1	Running	4 (10m ago)	3h21m
pod/kube-apiserver-minikube	1/1	Running	4 (10m ago)	3h21m
pod/kube-controller-manager-minikube	1/1	Running	4 (10m ago)	3h21m
pod/kube-proxy-5pg6s	1/1	Running	4 (10m ago)	3h21m
pod/kube-scheduler-minikube	1/1	Running	4 (10m ago)	3h21m
pod/metrics-server-85b7d694d7-8w2fp	1/1	Running	3 (10m ago)	93m
pod/storage-provisioner	1/1	Running	8 (10m ago)	3h21m

NAME	AGE	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/kube-dns	3h21m	ClusterIP	10.96.0.10	<none>	53/UDP,53/TCP,9153/TCP
service/metrics-server	93m	ClusterIP	10.109.130.77	<none>	443/TCP

Рисунок 3.17 — Pods и Services в namespace kube-system

Отключается дополнение ingress:

```
alexgribkov@i113971732 practical-work-5 % minikube addons disable ingress
🔍 "The 'ingress' addon is disabled"
```

Рисунок 3.18 — Отключение дополнения ingress

3.8 Работа с Kubernetes Dashboard

Kubernetes Dashboard предоставляет веб-интерфейс для управления кластером. Запускается Dashboard:

```
alexgribkov@i113971732 practical-work-5 % minikube addons disable ingress
🔴 "The 'ingress' addon is disabled
alexgribkov@i113971732 practical-work-5 % minikube dashboard
👉 Verifying dashboard health ...
🚀 Launching proxy ...
👉 Verifying proxy health ...
🚀 Opening http://127.0.0.1:64136/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...
```

Рисунок 3.19 — Запуск Kubernetes Dashboard

В Dashboard открывается раздел Deployments, который отображает следующие параметры:

- **Name** — имя Deployment (gribkov-ikbo-16-22)
- **Namespace** — пространство имен, в котором развернут Deployment (default)
- **Labels** — метки для идентификации и группировки ресурсов
- **Pods** — количество запущенных Pods и их статус
- **Replicas** — желаемое количество реплик (1/1)
- **Image** — используемый Docker-образ (gribkov-ikbo-16-22-obraz)
- **Age** — время существования Deployment
- **Selector** — селектор для выбора управляемых Pods (app=gribkov-ikbo-16-22)
- **Strategy** — стратегия обновления (RollingUpdate)
- **Events** — события, связанные с Deployment

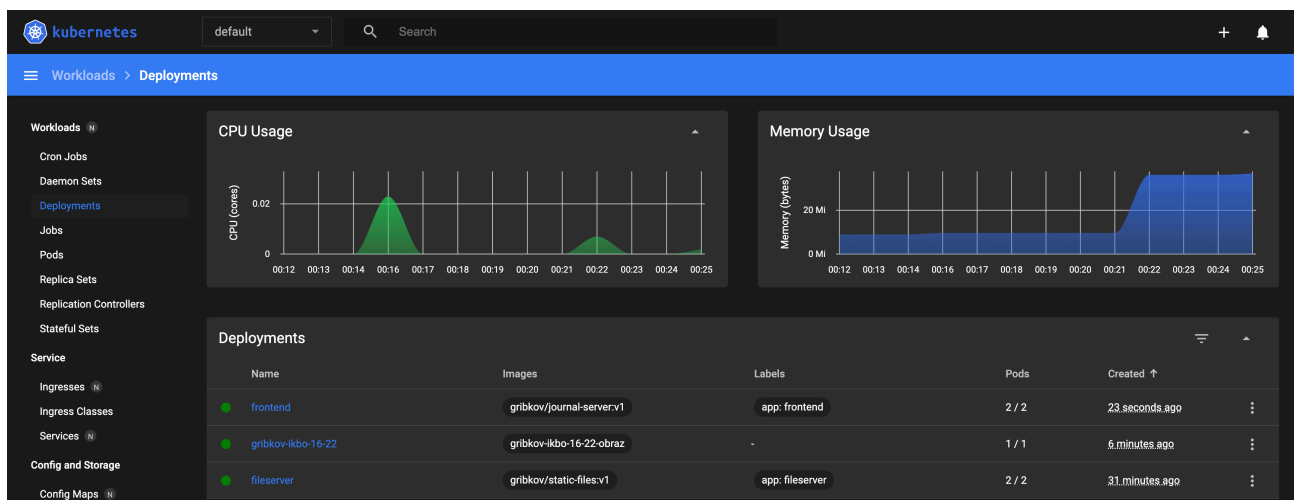


Рисунок 3.20 — Раздел Deployments в Kubernetes Dashboard

При открытии детальной информации о Deployment отображаются дополнительные параметры:

- **Pod Template** — шаблон для создания Pods, включая спецификацию контейнеров
- **Container Specifications** — детали контейнеров (имя, образ, порты, ресурсы)
- **Conditions** — условия состояния Deployment (Available, Progressing)
- **New Replica Set** — информация о ReplicaSet, управляющем текущими Pods
- **Resource Requests and Limits** — запросы и ограничения ресурсов (CPU, память)

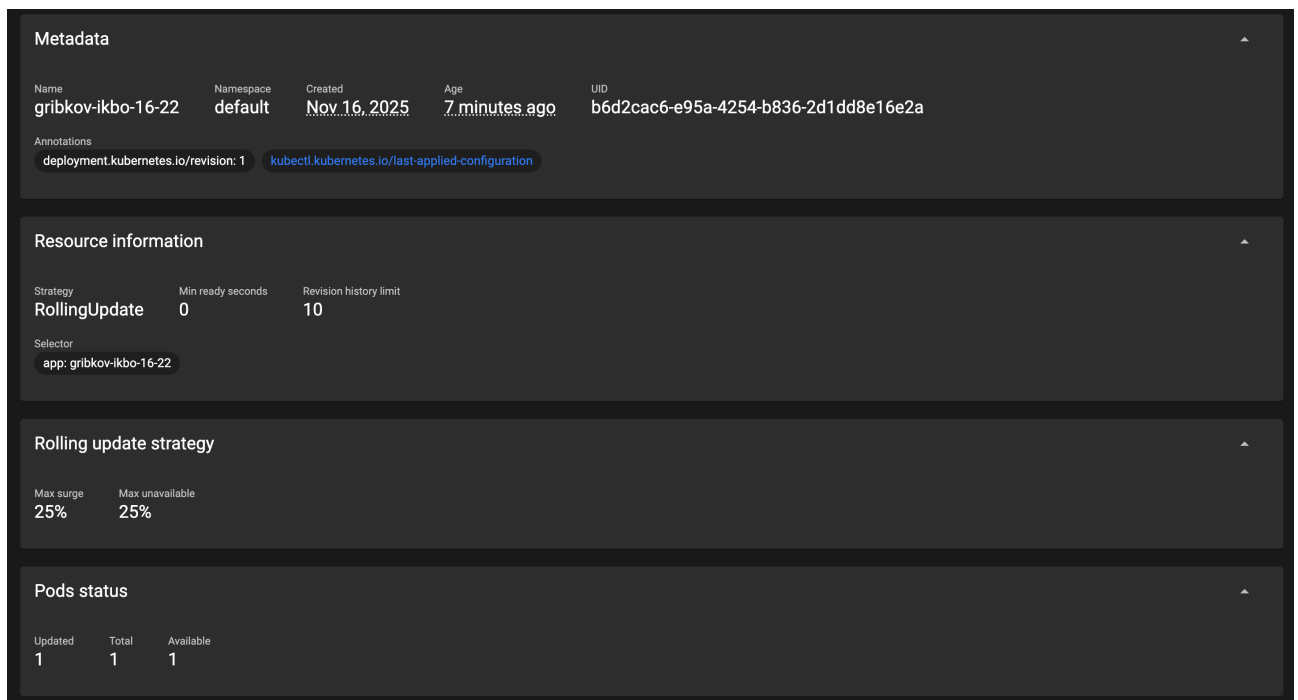


Рисунок 3.21 — Детальная информация о Deployment

3.9 Удаление ресурсов и остановка кластера

После завершения работы необходимо удалить созданные ресурсы. Удаляется Service:

```
alexgribkov@i113971732 practical-work-5 % kubectl delete service gribkov-ikbo-16-22
service "gribkov-ikbo-16-22" deleted from default namespace
```

Рисунок 3.22 — Удаление Service

Удаляется Deployment:

```
alexgribkov@i113971732 practical-work-5 % kubectl delete deployment gribkov-ikbo-16-22
deployment.apps "gribkov-ikbo-16-22" deleted from default namespace
```

Рисунок 3.23 — Удаление Deployment

Останавливается кластер Minikube:

```
alexgribkov@i113971732 practical-work-5 % kubectl delete deployment gribkov-ikbo-16-22
deployment.apps "gribkov-ikbo-16-22" deleted from default namespace
alexgribkov@i113971732 practical-work-5 % minikube stop
👉 Stopping node "minikube" ...
🔴 Powering off "minikube" via SSH ...
🔴 1 node stopped.
```

Рисунок 3.24 — Остановка кластера Minikube

4 ВЫВОД

В ходе выполнения практической работы были изучены основы работы с системой оркестрации контейнеров Kubernetes и инструментом локальной разработки Minikube.

Основные результаты работы:

1. Успешно установлен и настроен Minikube для запуска локального Kubernetes кластера.
2. Создано простое веб-приложение на Node.js и упаковано в Docker-образ с соблюдением требований по именованию.
3. Создан и применен манифест Deployment для развертывания приложения в кластере Kubernetes. Deployment обеспечивает декларативное управление приложением и его автоматическое восстановление при сбоях.
4. Освоены основные команды kubectl для управления кластером и просмотра его состояния, включая просмотр Deployments, Pods, событий и конфигурации.
5. Создан Service типа NodePort для обеспечения сетевого доступа к приложению извне кластера.
6. Изучена работа с дополнениями Minikube на примере ingress-контроллера, который позволяет настраивать маршрутизацию входящего трафика.
7. Освоен веб-интерфейс Kubernetes Dashboard, который предоставляет удобный способ мониторинга и управления ресурсами кластера.

Практическая работа продемонстрировала, как Kubernetes упрощает развертывание и управление контейнеризированными приложениями, обеспечивая автоматическое масштабирование, самовосстановление и декларативную конфигурацию. Minikube предоставляет удобную среду для изучения Kubernetes и разработки приложений без необходимости использования облачной инфраструктуры.

5 ОТВЕТЫ НА ВОПРОСЫ К ПРАКТИЧЕСКОЙ РАБОТЕ

5.1 Назовите виды контроллеров в Kubernetes

Контроллеры в Kubernetes — это управляющие циклы, которые отслеживают состояние кластера и при необходимости вносят изменения для достижения желаемого состояния.

Основные виды контроллеров:

1. **ReplicaSet** — обеспечивает поддержание указанного количества реплик Pods. Гарантирует, что определенное число идентичных Pods всегда запущено.
2. **Deployment** — предоставляет декларативные обновления для Pods и ReplicaSets. Управляет развертыванием новых версий приложений с возможностью отката.
3. **StatefulSet** — управляет развертыванием и масштабированием набора Pods с гарантией упорядоченности и уникальности. Используется для stateful приложений, таких как базы данных.
4. **DaemonSet** — обеспечивает запуск одной копии Pod на каждом узле кластера. Используется для системных сервисов, таких как сборщики логов или мониторинг.
5. **Job** — создает один или несколько Pods и обеспечивает успешное завершение указанного количества из них. Используется для пакетных задач.
6. **CronJob** — создает Jobs по расписанию, подобно cron в Unix-системах.
7. **ReplicationController** — устаревший контроллер, предшественник ReplicaSet. Гарантирует запуск указанного числа реплик Pods.

5.2 Как называется командная строка в Kubernetes?

Командная строка в Kubernetes называется **kubectl** (произносится как “cube-control” или “cube-cuttle”).

kubectl — это интерфейс командной строки для выполнения команд в кластерах Kubernetes. Он позволяет разворачивать приложения, проверять и управлять ресурсами кластера, просматривать логи и выполнять другие операции.

Основные возможности kubectl:

- Управление ресурсами (create, apply, delete, edit)
- Просмотр информации о ресурсах (get, describe, logs)
- Масштабирование приложений (scale)
- Обновление приложений (rollout)
- Отладка (exec, port-forward, attach)
- Управление конфигурацией (config)

kubectl взаимодействует с Kubernetes API сервером для выполнения операций в кластере. Конфигурация kubectl хранится в файле kubeconfig, который содержит информацию о кластерах, пользователях и контекстах.

5.3 Что такое под?

Pod (под) — это наименьшая и простейшая единица развертывания в Kubernetes, которая может быть создана и управляема.

Основные характеристики Pod:

1. **Группа контейнеров** — Pod может содержать один или несколько контейнеров, которые тесно связаны и должны работать вместе. Контейнеры в одном Pod разделяют общие ресурсы.

2. **Общая сеть** — все контейнеры в Pod разделяют один сетевой namespace, включая IP-адрес и порты. Контейнеры могут общаться друг с другом через localhost.
3. **Общее хранилище** — контейнеры в Pod могут использовать общие volumes для обмена данными.
4. **Эфемерность** — Pods являются эфемерными объектами. Они могут быть созданы, уничтожены и пересозданы по мере необходимости. При пересоздании Pod получает новый IP-адрес.
5. **Атомарность** — Pod является атомарной единицей планирования. Все контейнеры в Pod всегда размещаются на одном узле кластера.
6. **Жизненный цикл** — Pod проходит через различные фазы: Pending, Running, Succeeded, Failed, Unknown.

Pods обычно не создаются пользователями напрямую. Вместо этого используются контроллеры высокого уровня, такие как Deployment, которые управляют созданием и жизненным циклом Pods.

5.4 Назовите 2 типа ресурсов, из которых состоит кластер Kubernetes

Кластер Kubernetes состоит из двух основных типов ресурсов:

1. **Control Plane (Master Node)** — управляющий слой кластера, который принимает глобальные решения о кластере и обнаруживает события кластера.

Компоненты Control Plane:

- **kube-apiserver** — предоставляет Kubernetes API, является фронтендом для Control Plane. Все операции в кластере проходят через API сервер.
- **etcd** — распределенное key-value хранилище для хранения всех данных кластера, включая конфигурацию, состояние и метаданные.

- **kube-scheduler** — отслеживает вновь созданные Pods без назначенного узла и выбирает узел для их запуска на основе требований к ресурсам.
- **kube-controller-manager** — запускает процессы-контроллеры, которые регулируют состояние кластера (Node Controller, ReplicaSet Controller, Endpoints Controller и др.).
- **cloud-controller-manager** — взаимодействует с облачными провайдерами для управления облачными ресурсами.

2. **Worker Nodes (Node)** — рабочие узлы кластера, на которых выполняются контейнеризированные приложения.

Компоненты Worker Node:

- **kubelet** — агент, который выполняется на каждом узле кластера. Обеспечивает запуск контейнеров в Pods и следит за их работоспособностью.
- **kube-proxy** — сетевой прокси, который поддерживает сетевые правила на узлах. Реализует сетевую модель Service.
- **Container Runtime** — программное обеспечение для запуска контейнеров (Docker, containerd, CRI-O и др.).

Control Plane управляет Worker Nodes и Pods в кластере. В production-среде Control Plane обычно работает на нескольких узлах для обеспечения отказоустойчивости, а Worker Nodes выполняют приложения.

5.5 Чем Kubernetes отличается от Docker Swarm?

Kubernetes и Docker Swarm — это две системы оркестрации контейнеров, но они имеют существенные различия:

1. Сложность и кривая обучения

Docker Swarm проще в освоении и настройке. Он интегрирован в Docker и использует знакомый интерфейс командной строки Docker. Kubernetes имеет

более крутую кривую обучения из-за большего количества концепций и возможностей.

2. Масштабируемость

Kubernetes разработан для работы с очень большими кластерами (тысячи узлов) и обеспечивает более эффективное масштабирование. Docker Swarm лучше подходит для кластеров малого и среднего размера.

3. Функциональность

Kubernetes предоставляет значительно больше функций: автоматическое масштабирование (Horizontal Pod Autoscaler), более гибкое управление сетями, поддержка различных типов хранилищ, встроенная система мониторинга и логирования. Docker Swarm предоставляет базовые функции оркестрации.

4. Доступность и отказоустойчивость

Kubernetes имеет более продвинутые механизмы обеспечения высокой доступности, включая автоматическое перемещение Pods при сбоях узлов, health checks, self-healing. Docker Swarm также обеспечивает отказоустойчивость, но с меньшими возможностями настройки.

5. Экосистема

Kubernetes имеет обширную экосистему инструментов и поддержку всех основных облачных провайдеров (Google Kubernetes Engine, Amazon EKS, Azure AKS). Docker Swarm имеет меньшую экосистему и поддержку.

6. Развертывание приложений

Kubernetes использует декларативный подход с манифестами YAML, поддерживает сложные стратегии развертывания (rolling updates, blue-green, canary). Docker Swarm использует Docker Compose файлы и более простые стратегии развертывания.

7. Сетевая модель

Kubernetes использует CNI (Container Network Interface) и поддерживает различные сетевые плагины. Docker Swarm использует встроенную overlay-сеть, которая проще в настройке, но менее гибкая.

8. Сообщество и поддержка

Kubernetes имеет большое активное сообщество, регулярные обновления и поддержку от Cloud Native Computing Foundation. Docker Swarm имеет меньшее сообщество и более медленное развитие.

Выбор между Kubernetes и Docker Swarm:

Docker Swarm подходит для небольших проектов, когда требуется быстрое развертывание и простота управления. Kubernetes является лучшим выбором для enterprise-приложений, требующих высокой масштабируемости, гибкости и продвинутых возможностей оркестрации.

6 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Kubernetes Documentation. URL: <https://kubernetes.io/docs/home/> (дата обращения: 15.11.2025)
2. Minikube Documentation. URL: <https://minikube.sigs.k8s.io/docs/> (дата обращения: 15.11.2025)
3. kubectl Reference Documentation. URL: <https://kubernetes.io/docs/reference/kubectl/> (дата обращения: 15.11.2025)
4. Kubernetes Concepts. URL: <https://kubernetes.io/docs/concepts/> (дата обращения: 15.11.2025)
5. Docker Documentation. URL: <https://docs.docker.com/> (дата обращения: 15.11.2025)